

AegisPay — Error & Troubleshooting Guide

One document for every error hit while building AegisPay and how each was solved.
Start a fresh section whenever a new error appears, make sure to verify locally (lint + type + test) before pushing, catches most of these in seconds.
Conventions: each entry states the error, why it happened, the fix, and **the guard**** (how we stop it recurring).

A. Local tooling / Python environment

A1. ruff, mypy, pytest are "not recognized as ... a command"

- Error: The term 'ruff' is not recognized ...
- Why: The tooling is installed as a Python package but its Scripts dir is not on PATH (common on Windows when installed to the user site-packages).
- Fix: invoke via the module, or activate a venv whose scripts are on PATH:

```
python -m ruff check .
python -m mypy --config-file api/pyproject.toml
python -m pytest -q tests/unit
```
- Guard: CI uses `pip install -e "api[dev]"` which drops scripts on PATH on Linux; locally, prefer `python -m <tool>`.

A2. mypy: can't read file 'api/modules': No such file or directory

- Why: `[tool.mypy] files = ["api/modules", "api/policy"]` is relative to the `**config file**`, and CI/environment differs on the working directory. Running `cd api && mypy --config-file pyproject.toml` double-nests to `api/api/modules`.
- Fix: run mypy from the repo root:

```
mypy --config-file api/pyproject.toml
```

(This is exactly what `make type` now does.)
- Guard: the root Makefile type target runs from repo root; `api/Makefile` type does `cd .. && mypy --config-file api/pyproject.toml`.

A3. ModuleNotFoundError: No module named 'structlog'

- Why: importing `api.main` executes `configure_logging()` which imports `structlog`; the runtime deps were not installed in the active environment.
- Fix: install the project (dev extras) in the active interpreter:

```
pip install -e "api[dev]"
```
- Guard: phase 0 requires a reproducible env; CI installs `api[dev]`; the smoke test now imports `api.main` so a missing dep surfaces immediately.

B. Docker build / runtime

B1. app container crash: ModuleNotFoundError: No module named 'api'

- Why: the app build context was `api/` and the Dockerfile did `COPY . .` then `pip install .` But the Python package `api` lives at the repo root (e.g. `api/main.py` imports `api.core`). Copying the contents of `api/` into `/app` leaves the modules loose at `/app`, so `import api` fails. `pip install .` also found no `api` package to build -> empty install.
- Fix: build from the repo root and place the package in place before installing:

```
# compose: context: ../.. dockerfile: api/Dockerfile
COPY api/pyproject.toml ./
COPY api ./api
RUN pip install --no-cache-dir .
```

```
CMD ["uvicorn", "api.main:app", ...]
```

- Guard: `api/Dockerfile` and `api/Dockerfile.ai` both copy the package before ``pip install; docker compose config`` validates the wiring.

B2.db_migrate: connection to server at "db" ... Connection refused

- Why: `db_migrate` started before Postgres finished initialising (no healthcheck), so the first connection failed and the one-shot job exited.
- Fix: add a healthcheck to `db` and gate the migration on it:

```
db:
  healthcheck: { test: ["CMD-SHELL", "pg_isready -U aegispay_migration -d aegispay"], ... }
db_migrate:
  depends_on: { db: { condition: service_healthy } }
```

- Guard: `db` is now reported (healthy) in `docker compose ps` before the migration runs.

B3.db_migrate: psql: error: fe_sendauth: no password supplied

- Why: `psql` over TCP requires a password; the one-shot container had none set.
- Fix: provide it as `env` (and stop on first SQL error):

```
db_migrate:
  environment: { PGPASSWORD: aegispay }
  command: ["psql", "-v", "ON_ERROR_STOP=1", ...]
```

- Guard: migration now either succeeds (exit 0) or halts loudly on the first SQL error.

B4.localstack: Localstack returning with exit code 55 ... License activation failed

- Why: `localstack/localstack:latest` now requires an auth token/license; no credentials were present, so it quit.

- Fix: pin the free OSS image that does not license-gate:

```
localstack: { image: localstack/localstack:3.5.0 }
```

- Guard: pin a known-OSS tag instead of `latest` so a future tag does not introduce a license requirement.

B5.Docker build fails: pip ... ReadTimeoutError ... files.pythonhosted.org

- Why: the default `pip` timeout is too short on a slow network; the `pip` install inside the image download timed out (not a code error).

- Fix: raise the timeout in both Dockerfiles:

```
ENV PIP_DEFAULT_TIMEOUT=120 PIP_DISABLE_PIP_VERSION_CHECK=1
RUN pip install --no-cache-dir --timeout 120 .
```

- Guard: `PIP_DEFAULT_TIMEOUT` is now baked into the image build; retry if a transient network blip occurs.

C. CI / tests

C1.AttributeError: '_IncludedRouter' object has no attribute 'path'

- Error (CI, job test):

```
tests/unit/test_smoke.py:15: AttributeError
1 failed, 40 passed
```

- Why: the smoke test enumerated routes by iterating `app.routes` and reading `.path`. FastAPI's iteration contract for `app.routes` is private. The CI resolves `fastapi` to the latest (`0.141.1`), where included routers surface as an internal `_IncludedRouter` object with no `.path`. The dev box had older `fastapi` (`0.135.2`), so it passed locally and failed only in CI -> dependency version skew from unbounded `>=` ranges.

- Fix: stop introspecting `app.routes`; use the stable OpenAPI schema:

```
paths = set(app.openapi()["paths"])
```

```
assert "/v1/health" in paths
```

Also bound the range: `fastapi>=0.115, <0.142`.

- Guard: smoke test is now version-agnostic; FastAPI is bounded; reproduce CI locally with `pip install -e "api[dev]"`.

D. PDF generation (ReportLab)

D1. Parse error: saw `</font>` instead of expected `</i>`

- Why: inline code like `workers/*.py` contained a token `(*/_)` that the italic/bold regexes matched, producing malformed nested ReportLab markup.
- Fix: tokenise code spans with placeholders before applying emphasis, then restore:

```
text = re.sub(r"``([^\`]+)`", save_code, text)  # -> @@CODE0@@
text = escape(text)
text = re.sub(r"\*(...)\*", "<b>...</b>")      # bold
text = re.sub(r"\*([^\`]+)\*", "<i>...</i>")      # italic
# then replace @@CODE0@@ with <font name='Courier'>...</font>
```
- Guard: the shared `pdf/markdown_pdf.py` renderer always escapes code spans first.

D2. ModuleNotFoundError: No module named 'pdf'

- Why: running `python pdf/build_x.py` puts `pdf/` on `sys.path`, so `import pdf.markdown_pdf` cannot find the repo-root package.
- Fix: insert the repo root so the `pdf/api` packages resolve:

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```
- Guard: every build script adds the repo root to `sys.path` before importing the shared renderer.

E. Repository / docs

E1. Malformed Markdown code fence (single backtick) in `STRUCTURE.md`

- Why: the fenced code block was opened with a single backtick instead of three, so the tree rendered as plain text in some viewers.
- Fix: use triple backticks for the opening and closing fence.
- Guard: rules that detect a lone backtick line; render/verify the doc to confirm.

Prevention checklist (before every push)

- `[ ] make verify` green locally (lint + type + test) — 41 tests.
- `[ ] ruff format --check api clean` (100 files).
- `[ ]` Environment matches CI: `pip install -e "api[dev]"`.
- `[ ]` New code adds a test; the suite never shrinks.
- `[ ]` No secrets, no untracked build artifacts staged.
- `[ ] docker compose config` validates before committing compose changes.